

PSTAT 10 Data Science Principles

Lecture 15: Aliasing and Logicals

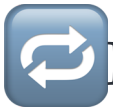
John Robin Inston 

johninston@ucsb.edu

University of California, Santa Barbara

August 25, 2026

Introduction



Review: Lecture 14



Last lecture we looked at:

- Relational databases
 - Data structure
 - Data integrity
 - Data manipulation
- Database connections
- Basic SQL queries

Outline: Lecture 15

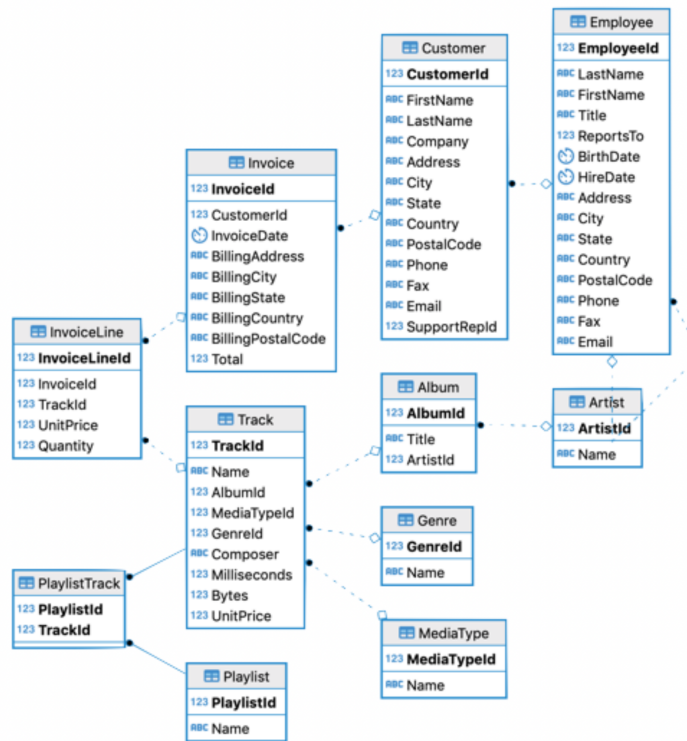
 Today we will emphasize querying and modifying data:

- Complex data selections
- SQL functions
- SQL syntax and convention

Chinook Database

Relation Network

We will continue working with the **Chinook** database with **structure**:



Relation Network

Each **arrow** represents a **foreign-key relationship** — it shows how a column in one table **references** the primary key of another, letting us connect data across tables.

Connecting

Last time we learned how to connect to an **external database**. Make sure you have the appropriate **libraries** loaded.

```
1 library(DBI)
2 library(RSQLite)
3 library(sqldf)
4 library(tidyverse)
```

Then create an appropriate **database object**:

```
1 chinook_db <- dbConnect(
2   SQLite(),
3   "data/Chinook_Sqlite.sqlite"
4 )
```

Things to watch out for:

- If you do not have an appropriate file path pointing to the `.sqlite` file, R will populate an **empty database** object.
- If you wish to avoid working with file paths, place the `.sqlite` file in your current working directory.



SQL Query Structure

Recall our **basic SQL query** from last lecture that had the following **structure**:

- 1 SELECT attributes
- 2 FROM relations
- 3 WHERE conditions

We also saw the **INSERT** and **LIMIT** arguments in these SQL queries.

```
1 chinook_db |>
2 dbGetQuery(
3   "SELECT InvoiceId, BillingCity
4   FROM Invoice
5   LIMIT 5"
6 )
```

	InvoiceId	BillingCity
1	1	Stuttgart
2	2	Oslo
3	3	Brussels
4	4	Edmonton
5	5	Boston

Here is a second example using **WHERE** and a different relation:

```
1 chinook_db |>
2 dbGetQuery(
3   "SELECT Name, AlbumId, GenreId
4   FROM Track
5   WHERE AlbumId = 3 AND GenreId = 1"
6 )
```

	Name	AlbumId	GenreId
1	Fast As a Shark	3	1
2	Restless and Wild	3	1
3	Princess of the Dawn	3	1

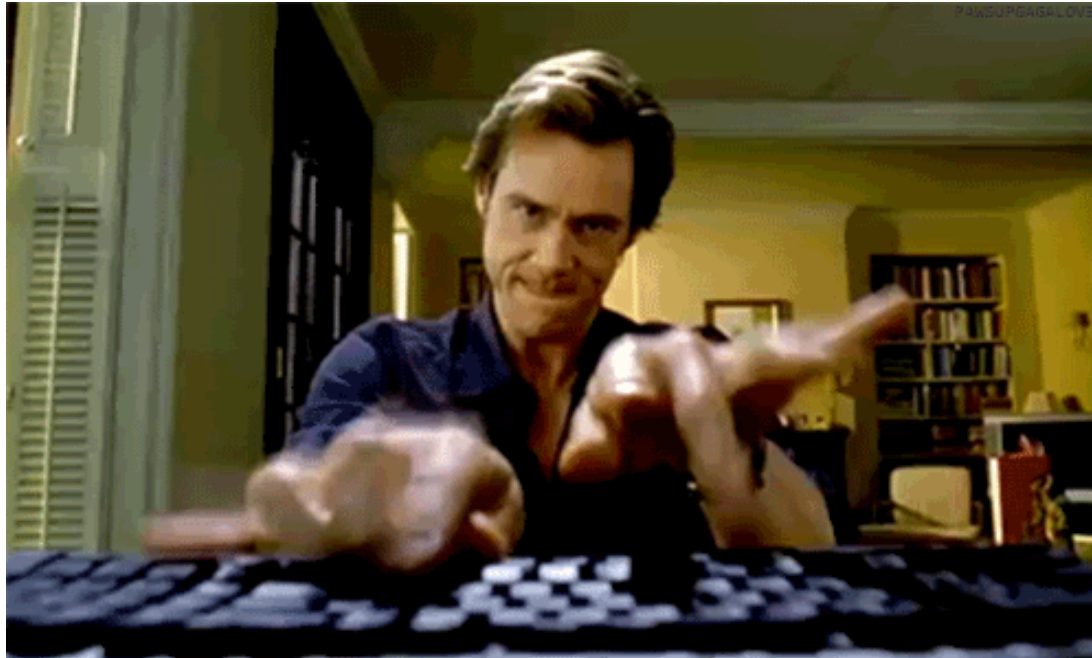
Specialized Selection

Suppose we wanted to **select all attributes** of the `invoice` table.

```
1 chinook_db |>  
2 dbListFields("invoice")
```

[1]	"InvoiceId"	"CustomerId"	"InvoiceDate"
[4]	"BillingAddress"	"BillingCity"	"BillingState"
[7]	"BillingCountry"	"BillingPostalCode"	"Total"

!! This is going to be a **hefty query** — but we know how to write it!



So get typing!

★ Selecting All Attributes



Too Many Attributes

Written out in full we have:

```
1 chinook_db |> dbGetQuery(  
2   "SELECT InvoiceId, CustomerId, InvoiceDate,  
3     BillingAddress, BillingCity, BillingState,  
4     BillingCountry, BillingPostalCode, Total  
5   FROM invoice  
6   LIMIT 2"  
7 )
```

InvoiceId	CustomerId	InvoiceDate	BillingAddress
1	1	2 2009-01-01 00:00:00	Theodor-Heuss-Straße 34 Stuttgart
2	2	4 2009-01-02 00:00:00	Ullevålsveien 14 Oslo
			BillingState BillingCountry BillingPostalCode Total
1	<NA>	Germany	70174 1.98
2	<NA>	Norway	0171 3.96

* Syntax

Instead, to **select all attributes** we can use the * symbol:

```
1 chinook_db |> dbGetQuery(  
2   "SELECT *  
3   FROM Invoice  
4   LIMIT 2"  
5 )
```

InvoiceId	CustomerId	InvoiceDate	BillingAddress
1	1	2 2009-01-01 00:00:00	Theodor-Heuss-Straße 34 Stuttgart
2	2	4 2009-01-02 00:00:00	Ullevålsveien 14 Oslo
			BillingState BillingCountry BillingPostalCode Total
1	<NA>	Germany	70174 1.98
2	<NA>	Norway	0171 3.96



When in doubt, keep your queries **simple!** 👍

Combining SQL and R

Using `select()`

Sometimes it may be helpful to **combine functionality** across **different languages**:

- Load `tidyverse` if you have not already done so; and
- Load `sqldf` if you have not already done so.

```
1 chinook_db |>
2   dbGetQuery(
3     "SELECT *
4     FROM Invoice"
5   ) |>
6   select(InvoiceId, BillingCity, Total) |>
7   str()
```

```
'data.frame':   412 obs. of  3 variables:
 $ InvoiceId  : int  1 2 3 4 5 6 7 8 9 10 ...
 $ BillingCity: chr  "Stuttgart" "Oslo" "Brussels" "Edmonton" ...
 $ Total     : num  1.98 3.96 5.94 8.91 13.86 ...
```

Remember, `dbGetQuery()` returns the query result as a **dataframe**:

- Here we **piped** the `select()` function to select columns.
- Then we used the structure `str()` function to examine our results.

Querying Data

We have used **WHERE** before...

The syntax **WHERE** allows us to work with **logicals**. Make sure to think carefully about the logic you are trying to implement.

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCity, BillingCountry, Total  
3     FROM Invoice WHERE Total > 20"  
4 )
```

	BillingCity	BillingCountry	Total
1	Budapest	Hungary	21.86
2	Dublin	Ireland	21.86
3	Fort Worth	USA	23.86
4	Prague	Czech Republic	25.86

Logicals in SQL

The syntax **AND** and **OR** allow us to select **intersections** and **unions** respectively:

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCity, BillingCountry, Total  
3     FROM Invoice  
4     WHERE Total > 20 AND BillingCity = 'Budapest'"  
5 )
```

	BillingCity	BillingCountry	Total
1	Budapest	Hungary	21.86

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCity, BillingCountry, Total  
3     FROM Invoice  
4     WHERE Total > 20 OR BillingCountry = 'Italy'  
5     LIMIT 3"  
6 )
```

	BillingCity	BillingCountry	Total
1	Rome	Italy	1.98
2	Rome	Italy	3.96
3	Budapest	Hungary	21.86

Combining Logicals

Combine clauses similarly to combining R logical expressions.

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCity, BillingCountry, Total  
3   FROM Invoice  
4   WHERE Total > 20 OR BillingCountry = 'Italy'  
5   AND BillingCity = 'Rome'  
6   LIMIT 4"  
7 )
```

	BillingCity	BillingCountry	Total
1	Rome	Italy	1.98
2	Rome	Italy	3.96
3	Budapest	Hungary	21.86
4	Rome	Italy	5.94

- This query returns all invoices with a total greater than 20, or all invoices from Italy where the billing city is Rome.
 - Notice the **order of operations** — **AND** is evaluated before **OR**.

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCity, BillingCountry, Total  
3   FROM Invoice  
4   WHERE (Total > 20 OR BillingCountry = 'Italy')  
5   AND BillingCity = 'Rome'  
6   LIMIT 6"  
7 )
```

	BillingCity	BillingCountry	Total
1	Rome	Italy	1.98
2	Rome	Italy	3.96
3	Rome	Italy	5.94
4	Rome	Italy	0.99
5	Rome	Italy	1.98
6	Rome	Italy	13.86

- With the parentheses, the query returns all invoices from Rome where the total is greater than 20 or the billing country is Italy.

Exclusion

Sometimes we wish to **exclude** specific values. Suppose we wanted all invoices **where** the total is NOT .99:

```
1 chinook_db |>
2   dbGetQuery(
3     "SELECT BillingCity, BillingCountry, Total
4     FROM Invoice
5     WHERE NOT Total = .99
6     LIMIT 5"
7   )
```

	BillingCity	BillingCountry	Total
1	Stuttgart	Germany	1.98
2	Oslo	Norway	3.96
3	Brussels	Belgium	5.94
4	Edmonton	Canada	8.91
5	Boston	USA	13.86

This **NOT** syntax is **very powerful** when **combined** with the **AND**, **OR** commands.

```
1 dbGetQuery(
2   chinook_db,
3   "SELECT BillingCity, BillingCountry, Total
4   FROM Invoice
5   WHERE BillingCountry = 'USA'
6   LIMIT 3"
7 )
```

	BillingCity	BillingCountry	Total
1	Boston	USA	13.86
2	Mountain View	USA	0.99
3	Redmond	USA	1.98

```
1 dbGetQuery(
2   chinook_db,
3   "SELECT BillingCity, BillingCountry, Total
4   FROM Invoice
5   WHERE BillingCountry = 'USA'
6   AND NOT BillingCity = 'Boston'
7   LIMIT 3"
8 )
```

	BillingCity	BillingCountry	Total
1	Mountain View	USA	0.99
2	Redmond	USA	1.98
3	Cupertino	USA	1.98

Logical Syntax

SQL tries to closely **mimic human language**:

- `AND`, `OR`, `NOT` used rather than `&`, `|`, `!`
- The `NOT` (**negated clause**) operator breaks (English) language convention slightly. SQL also allows the `<>` operator to mean **not equal**:

```
1 dbGetQuery(chinook_db,  
2   "SELECT BillingCity, BillingCountry, Total  
3     FROM Invoice  
4     WHERE BillingCountry = 'USA'  
5       AND BillingCity <> 'Boston'  
6     LIMIT 1"  
7 )
```

BillingCity	BillingCountry	Total
1 Mountain View	USA	0.99

 The syntax `!=` does work in SQLite but not universally. Please use the `<>` convention whenever possible.



Exercise — Involved Queries

Write and execute a SQL query that retrieves the `InvoiceId`, `InvoiceDate`, `BillingCity`, `BillingState` and `Total` for all invoices of customers based in the `United States` who spent between `$15` and `$20`.

04:00



Solution — Involved Queries

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT InvoiceId, InvoiceDate, BillingCity, BillingSta  
4     FROM Invoice  
5     WHERE BillingCountry = 'USA'  
6     AND Total >= 15 AND Total <= 20"  
7 )
```

	InvoiceId	InvoiceDate	BillingCity	BillingState	Total
1	103	2010-03-21 00:00:00	Chicago	IL	15.86
2	201	2011-05-29 00:00:00	Madison	WI	18.86

This query works, i.e. it returns the desired values — but:

- We want to make queries as **simple as possible**.
- Use available functionality — **BETWEEN** is helpful.



Better Solution — Involved Queries

A better solution uses `BETWEEN`:

```
1 chinook_db |> dbGetQuery(  
2   "SELECT InvoiceId, InvoiceDate, BillingCity, BillingSta  
3     FROM Invoice  
4     WHERE BillingCountry = 'USA'  
5     AND Total BETWEEN 15 AND 20"  
6 )
```

	InvoiceId	InvoiceDate	BillingCity	BillingState	Total
1	103	2010-03-21 00:00:00	Chicago	IL	15.86
2	201	2011-05-29 00:00:00	Madison	WI	18.86

Why is this better?

- `BETWEEN` is **inclusive** of both bounds — equivalent to `>= 15 AND <= 20`.
- It replaces **two comparisons** with a **single clause**.
- It reads almost exactly like the prompt: “**between** `$15` and `$20`”.

Table Exploration

It is often useful to know which attribute values exist in a table.

- `SELECT DISTINCT` allows us to pick only **unique values**.

```
1 chinook_db |>
2 dbGetQuery(
3   "SELECT DISTINCT BillingCountry
4   FROM Invoice
5   LIMIT 5"
6 )
```

	BillingCountry
1	Germany
2	Norway
3	Belgium
4	Canada
5	USA

Distinct Combinations

- **Combining** attribute values looks for **distinct combinations**.

```
1 dbGetQuery(
2   chinook_db,
3   "SELECT DISTINCT BillingCountry,
4   BillingState
5   FROM Invoice
6   LIMIT 5"
7 )
```

	BillingCountry	BillingState
1	Germany	<NA>
2	Norway	<NA>
3	Belgium	<NA>
4	Canada	AB
5	USA	MA

12 34 Counting and Ordering

Counting Unique Values

Sometimes we wish to know the **number of unique values** using the syntax **COUNT** :

```
1 chinook_db |> dbGetQuery(  
2   "SELECT COUNT(DISTINCT BillingCountry)  
3   FROM Invoice"  
4 )
```

```
COUNT(DISTINCT BillingCountry)  
1                                24
```

COUNT can also be used independently of **DISTINCT** :

```
1 chinook_db |> dbGetQuery(  
2   "SELECT COUNT(*)  
3   FROM Invoice"  
4 )
```

```
COUNT(*)  
1         412
```

Ordering Values

Suppose we want to see the invoice totals **ordered** from **minimum** to **maximum**. We can use the **ORDER BY** query.

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCountry, BillingCity, Total  
3   FROM Invoice  
4   ORDER BY Total  
5   LIMIT 3"  
6 )
```

	BillingCountry	BillingCity	Total
1	Germany	Frankfurt	0.99
2	USA	Mountain View	0.99
3	United Kingdom	Edinburgh	0.99



Descending Order, Minimum and Maximum

Sorting in Descending Order

To sort in **descending** order we would write:

```
1 chinook_db |>  
2 dbGetQuery(  
3   "SELECT BillingCountry, BillingCity, Total  
4   FROM Invoice  
5   ORDER BY Total DESC  
6   LIMIT 6"  
7 )
```

	BillingCountry	BillingCity	Total
1	Czech Republic	Prague	25.86
2	USA	Fort Worth	23.86
3	Hungary	Budapest	21.86
4	Ireland	Dublin	21.86
5	Austria	Vienne	18.86
6	USA	Madison	18.86



MIN() and MAX() Functions

The **MIN()** and **MAX()** functions isolate the **smallest** and **largest** values:

```
1 chinook_db |> dbGetQuery(  
2   "SELECT BillingCountry, BillingCity, MIN(Total)  
3   FROM Invoice"  
4 )
```

	BillingCountry	BillingCity	MIN(Total)
1	Germany	Frankfurt	0.99

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT BillingCountry, BillingCity, MAX(Total)  
4   FROM Invoice"  
5 )
```

	BillingCountry	BillingCity	MAX(Total)
1	Czech Republic	Prague	25.86



Note the behavior in the case of ties.



Exercise — Multifaceted Queries

Write and execute an SQL query that:

1. Pulls the `TrackId`, `Name`, `AlbumId`, `GenreId` attributes of `Track`.
2. Only returns tracks **between** 200 and 300 seconds (200000 and 300000 milliseconds) long.
3. Only returns tracks **between** 2000000 and 5000000 bytes.
4. Returns tracks in **descending order** by **price**.
5. Prints no more than 5 rows to the console.

05:00



Solution — Multifaceted Queries

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT TrackId, Name, AlbumId, GenreId  
4   FROM Track  
5   WHERE Milliseconds BETWEEN 200000 AND 300000  
6     AND Bytes Between 2000000 AND 5000000  
7   ORDER BY UnitPrice DESC  
8   LIMIT 5"  
9 )
```

	TrackId	Name	AlbumId	GenreId
1	3	Fast As a Shark	3	1
2	4	Restless and Wild	3	1
3	93	Exploder	10	1
4	94	Hypnotize	10	1
5	1146	Welcome to the Jungle	90	1

Breaking it down:

- Two **BETWEEN** filters joined with **AND** — one on **duration** (**Milliseconds**), one on **size** (**Bytes**).
- **ORDER BY UnitPrice DESC** returns tracks in **descending order** by price.
- **LIMIT 5** caps the output at **five rows**.



Bug or Feature

Error messages are good. Odd returns are much worse!

- Errors direct you to a specific problem.
- Errors are google-able and often immediately fixable.

What's happening here?

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT 'ucsb'  
4   FROM Track  
5   WHERE AlbumId = 3"  
6 )
```

```
'ucsb'  
1   ucsb  
2   ucsb  
3   ucsb
```



This is **not an error**: - `SELECT 'ucsb'` returns the **string literal** `'ucsb'` for **every matched row**. - `SELECT` can return more than just column values.

Understanding SELECT

SELECT is an **incredibly flexible** tool!

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT Name, AlbumId FROM Track WHERE AlbumId = 3"  
4 )  
5 dbGetQuery(  
6   chinook_db,  
7   "SELECT 'ucsb', Name, AlbumId FROM Track WHERE AlbumId = 3"  
8 )
```

	Name	AlbumId
1	Fast As a Shark	3
2	Restless and Wild	3
3	Princess of the Dawn	3

	'ucsb'	Name	AlbumId
1	ucsb	Fast As a Shark	3
2	ucsb	Restless and Wild	3
3	ucsb	Princess of the Dawn	3

SELECT can also return **computed values**:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT 'ucsb', power(2, 5), Name FROM Track WHERE AlbumId = 3"  
4 )  
5 dbGetQuery(  
6   chinook_db,  
7   "SELECT Bytes / Milliseconds, Name FROM Track WHERE AlbumId = 3"  
8 )
```

	'ucsb'	power(2, 5)	Name
1	ucsb	32	Fast As a Shark
2	ucsb	32	Restless and Wild
3	ucsb	32	Princess of the Dawn

	Bytes / Milliseconds	Name
1	17	Fast As a Shark
2	17	Restless and Wild
3	16	Princess of the Dawn

Aliasing

DEFINITION

Aliasing assigns a **temporary name** to a column (or table) that exists for the **duration of the query only**.

```
1 dbGetQuery(chinook_db,"SELECT Name AS Good_Music FROM Tra
```

```
                                Good_Music  
1 For Those About To Rock (We Salute You)  
2      Balls to the Wall
```

- Aliasing **does not change column names**; they exist for the query only.
- A **table** can also be aliased with the `FROM table AS alias` syntax.
- **Query-defined columns** can also be aliased.
- `WHERE` clauses can **reference columns by alias**.

Pattern Selection

Sometimes we don't know quite what we're looking for.

- Real-world data is often **messy**.
- Searching for **exact matches** often misses things.
- **Pattern matching** lets us search for **partial** or **approximate** values.

SQLite gives us two **pattern-matching operators**:

- **LIKE** — simple wildcards (**%**, **_**), **case-insensitive**.
- **GLOB** — Unix-style wildcards (*****, **?**), **case-sensitive**.

 Reach for **LIKE** first — it is **simpler** and covers most cases. Use **GLOB** when you need **case sensitivity** or **finer control**.



First Letter Selection

Selection based on **first letter** only:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT BillingCountry, BillingState, Total  
4     FROM Invoice  
5     WHERE BillingCountry LIKE 'u%'  
6     LIMIT 5"  
7 )
```

	BillingCountry	BillingState	Total
1	USA	MA	13.86
2	United Kingdom	<NA>	8.91
3	USA	CA	0.99
4	USA	WA	1.98
5	USA	CA	1.98



LIKE — Intermediate Strings

Selection based on **intermediate string**:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT BillingCountry, BillingState, Total  
4     FROM Invoice  
5     WHERE BillingCountry LIKE '%an%'  
6     LIMIT 5"  
7 )
```

	BillingCountry	BillingState	Total
1	Germany	<NA>	1.98
2	Canada	AB	8.91
3	Germany	<NA>	0.99
4	Germany	<NA>	1.98
5	France	<NA>	1.98



GLOB Precision

DEFINITION

SQLite's **GLOB** operator matches text values against a pattern using **wildcards** — ***** for any sequence of characters and **?** for a single character.

GLOB allows further precision and refinement in these queries:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT TrackId, Name  
4   FROM Track  
5   WHERE Name GLOB '?ere*'  
6   LIMIT 5"  
7 )
```

	TrackId	Name
1	324	Pererê
2	1132	Serenity
3	1452	Were Do We Go From Here
4	1740	Sereia
5	2198	Jeremy



Exercise — Combining Techniques

Write and execute an SQL query that:

1. Retrieves the `TrackId` and `Name` of all tracks with `AlbumId = 30`.
2. Also retrieves the length of the song in seconds (ms/1000) in a column called `Seconds`.
3. Also retrieves the size of the song in megabytes ($1\text{MB} = 10^{-6}$ bytes) in a column called `MB`.
4. Lists the tracks in ascending order by length (seconds).
5. Only includes songs containing the word `and` in their names.

05:00



Solution — Combining Techniques

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT TrackId, Name, Milliseconds/1000 AS Seconds, By  
4   FROM Track  
5   WHERE AlbumId = 30 AND Name LIKE '% and %'  
6   ORDER BY Seconds ASC")
```

	TrackId	Name	Seconds	MB
1	342	What is and Should Never Be	260	8.497116
2	340	Dazed and Confused	401	13.035765

Breaking it down:

- We compute **new columns** inside `SELECT` with arithmetic and **alias** them — `Milliseconds/1000 AS Seconds` and `Bytes*power(10, -6) AS MB`.
- `LIKE '% and %'` uses **surrounding spaces** to match the **word** “and”, not substrings like “land”.
- `ORDER BY Seconds ASC` sorts by the **alias** we just defined.

Styling SQL

Much like R, SQL code works without **proper styling**.

- It can be **extremely** tough to read.
- SQL is **designed for readability** — don't squander it!

Indentation should follow logical functions:

```
1 SELECT item_1, item_2, ... , item_k
2 FROM table_1 WHERE logical_1 AND OR logical_2 ... AND OR logical_m
3 ORDER BY order_1, ... , order_n ASC/DESC
```



Specific indentation is subjective, but your focus should be to make your code as **readable as possible**.

Summary



Topics Covered



This lecture covered many aspects of SQL code and syntax:

- **Logical operations** with `WHERE`
- Functionality associated with `SELECT`
- **Aliasing**
- Various other functions / operators (`LIMIT`, `ORDER BY`, `*`, etc.)



In general you shouldn't focus on memorizing syntax — instead focus on understanding it and using it to solve problems.



Next Class



Next class we will almost entirely focus on two topics:

- Aggregation (continuing from `COUNT`, `MIN`, `MAX`, etc.)
- **Joins**

Then, in lectures 17 and 18, we will cover a selection of more advanced SQL topics:

- Nested `SELECT` statements
- Table creation and data insertion
- Database design